

BINDER

Product Document & Source of Truth

Version 1.0 | April 2025 | Internal use only

This document is the single source of truth for the Binder product. It records all product, experience, and architecture decisions made to date. It must be consulted before any implementation decision is made.

Table of Contents

01	The Manifesto	The philosophy and reason for being
02	Product Decisions	What the product is and is not
03	Platform & Persistence	Stack and data strategy
04	The Scryfall API	External data source for sets and cards
05	Environment: Home	Binder listing and management
06	Environment: Binder	The collection in progress
07	Data Model	Supabase schema, constraints, indexes, RLS
08	Scope & Future Work	Deliberate exclusions and forward compatibility

The Manifesto

Collecting is an act of intention.

When someone decides to collect something, they are not merely accumulating objects. They are building a personal system of meaning — a narrative with a beginning, a measurable progression, and a horizon that keeps moving. The physical binder has always been the perfect artifact for that: visible, organized, *yours*.

"I know what I want to complete. I know what I already have. But I do not know exactly where I stand in that journey — and when I do, it requires paper, a spreadsheet, or memory."

The gap between a collector's intention and clarity of progress is the product. Binder exists to close that gap with the least possible friction.

The Three Principles

01 Ownership without friction

Marking that you have a card must cost no more than a single tap. Friction kills habits. If logging a new acquisition is effortful, the person stops logging — and the product loses its purpose.

02 The collection commands, the user responds

The structure of a collection is not invented by the user — it comes from the outside (the Scryfall API). The user does not organize; they react: have it or do not have it. The order is canonical. User control comes later, not now.

03 The binder is identity, not just data

Name and color are not metadata — they are personalization. They are the difference between 'I have a tracker' and 'this is my Murders at Karlov Manor binder'. That matters even without a login, even as local data. Especially as local data — because here, only you see it.

Product Decisions

What the product is

A web application that mirrors a physical card binder. The user creates binders, each bound to a specific Magic: The Gathering set, and marks which cards they already own. The focus is clarity of progress and simplicity of interaction.

What the product is not (and must not try to be now)

Price inventory	No market values, no cost tracking
Social network	No public profiles, no following other collectors
Marketplace	No buying, selling, or trading
Multi-device sync	No cross-device synchronization in this phase
Shareable tool	No public view links

The entire business logic in three questions

All product logic fits in three questions. Any feature that does not answer one of these must be challenged before being built:

Which binders do I have?	Answered by the Home screen
What is in this binder?	Answered by the binder detail page
Do I own this card?	Answered by the owned/not-owned toggle

Platform & Persistence

Stack decisions

Platform	Web (browser)
Database	Supabase (PostgreSQL 15)
Initial env	Local development — no deployment required at start
Authentication	Absent in this phase — no login, no user session
Sharing	Not available in this phase

Why Supabase from day one?

The decision to use Supabase — instead of `localStorage` — was made deliberately, even though login is out of scope for now. The reasoning:

- Login is coming. Modeling the schema correctly now avoids a full rewrite later.
- Tables can include `user_id` from the start without enforcing auth — the column exists, unused.
- `localStorage` is fragile: clearing browser data destroys everything. Supabase persists reliably.
- The cost of setup is roughly 30 minutes — disproportionately cheap compared to migrating later.
- Supabase ships with Row Level Security (RLS), the exact mechanism needed for multi-user isolation when auth lands.

Approach comparison

	<code>localStorage</code>	Supabase (now)
Data survives browser clear	No	Yes
Works offline	Always	Depends
Access from another device	No	Yes (future)
Requires login	No	No (for now)
Ready for auth without rewrite	No	Yes
Initial setup	Zero	~30 minutes

The Scryfall API

All set and card data comes from the Scryfall API — a free, public API with complete coverage of Magic: The Gathering. No API key is required.

Base URL	https://api.scryfall.com
Documentation	https://scryfall.com/docs/api
Auth	None — fully public
Rate limit	10 requests/second recommended
Format	JSON
Pagination	Cursor-based via <code>has_more</code> + <code>next_page</code> fields

Endpoints used by Binder

List sets	GET <code>/sets</code>	Populates the set selector when creating a binder
List cards	GET <code>/cards/search</code>	Loads all cards of a set in canonical order

Card search query parameters

When loading cards for a binder, the following parameters are used to ensure canonical ordering and complete set coverage:

SCRYFALL CARD QUERY

```
GET /cards/search
?q=set:{set_code}
■=set
&dir;=asc
&unique;=prints
&page;={n} // iterated until has_more = false
```

Fields consumed per card object

From each card object returned by the API, Binder reads only the following fields. All other fields (prices, legality, rulings, keywords, etc.) are discarded at parse time.

id	Scryfall UUID — used as the canonical card identifier in the database
-----------	---

name	Display name shown in the UI
collector_number	Position within the set — determines canonical sort order

Environment: Home

The Home screen is the product's entry point. It lists all created binders and exposes the primary management actions.

Empty state

When no binders exist yet, the Home renders an inviting empty state — with a clear call to create the first binder. This is not an error screen; it is an onboarding screen.

Binder listing

Each binder card in the Home displays:

- Binder name (prominent)
- Associated color (one of the 5 Magic colors)
- Linked set name
- Progress counter: X of Y cards

Sort order: **most recently created first** (by created_at descending). No manual reordering in this phase.

Creating a binder

The creation flow requires exactly three inputs:

Name	Required. Maximum 40 characters.
Color	Required. One of the 5 Magic colors: white, blue, black, red, green.
Set	Required. Dropdown populated by Scryfall GET /sets.

On confirmation, the binder appears immediately at the top of the list. All cards from the set are loaded with owned = false by default.

Binder actions

The following actions are available both from the Home and from within the binder page:

Rename	Edits the binder name. No confirmation required — reversible action.
Change color	Updates the associated color. No confirmation required.
Delete	Removes the binder and all card ownership records. Requires explicit confirmation. Irreversible — no trash, no undo.

Environment: Binder

The binder page is where the core experience happens. The user sees the full set and records what they own.

Header

The page header displays:

- Binder name — dominant visual element
- Linked set name
- Color as a visual design element
- Progress counter: X of Y cards (implied: Y-X remaining)
- Access to rename, change color, and delete actions

The counter updates in real time as cards are marked or unmarked.

The card list

Cards are rendered in canonical set order (collector_number from Scryfall, ascending). No user reordering in this phase.

Each card displays:

- Card name
- Collector number within the set

Marking and unmarking

The central interaction of the product. It must be immediate, clear, and frictionless:

Default state	All cards start as owned = false when the binder is created
Mark as owned	Single click/tap toggles to owned = true
Unmark	Second click/tap returns to owned = false
Visual feedback	Clear and immediate visual distinction between the two states
Persistence	Auto-saved to Supabase on every toggle — no save button
Quantity	Not applicable. One card is one card. No copy multiplicity.

Filter

A simple three-state filter is available within the binder page:

All	Shows all cards in the set (default)
Owned	Shows only cards marked as owned
Not owned	Shows only cards not yet owned

Data Model

The Supabase database consists of two tables. The schema is intentionally minimal: it captures only what is necessary to run the current product while remaining fully compatible with future multi-user auth without any structural change.

Table: binders

Stores one row per binder created by the user. Each binder is bound to exactly one Scryfall set and one of the five Magic colors.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()	Surrogate key. Generated by PostgreSQL.
name	text	NOT NULL CHECK (char_length(name) <= 40)	User-defined name. Max 40 characters.
color	text	NOT NULL CHECK (color IN (...))	One of: white, blue, black, red, green.
set_code	text	NOT NULL	Scryfall set code (e.g. mkm). Used to query cards.
set_name	text	NOT NULL	Human-readable set name cached at creation time.
created_at	timestampz	NOT NULL DEFAULT now()	Creation timestamp. Used for Home sort order (DESC).
user_id	uuid	REFERENCES auth.users(id)	NULL for now. Activated when auth is introduced.

DDL — binders

```
CREATE TABLE binders (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  name text NOT NULL CHECK (char_length(name) <= 40),  
  color text NOT NULL CHECK (color IN  
    ('white', 'blue', 'black', 'red', 'green')),  
  set_code text NOT NULL,  
  set_name text NOT NULL,
```

```

created_at timestamptz NOT NULL DEFAULT now(),
user_id uuid REFERENCES auth.users(id)
);

-- Index for Home screen default sort
CREATE INDEX binders_created_at_idx ON binders (created_at DESC);

-- Index for future user-scoped queries
CREATE INDEX binders_user_id_idx ON binders (user_id);

```

Table: card_ownership

Stores one row per card per binder. A row is created for every card in the set when a binder is first opened, with owned = false. The user's single interaction — the toggle — updates this boolean.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()	Surrogate key.
binder_id	uuid	NOT NULL REFERENCES binders(id) ON DELETE CASCADE	FK to binders. Cascade ensures no orphan rows.
card_id	text	NOT NULL	Scryfall card UUID. Source of truth for card identity.
collector_number	text	NOT NULL	Used to sort cards in canonical set order.
owned	boolean	NOT NULL DEFAULT false	true = owned, false = not owned.
user_id	uuid	REFERENCES auth.users(id)	NULL for now. Mirrors binders.user_id for RLS symmetry.

DDL — card_ownership

```

CREATE TABLE card_ownership (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  binder_id uuid NOT NULL
  REFERENCES binders(id) ON DELETE CASCADE,
  card_id text NOT NULL,
  collector_number text NOT NULL,
  owned boolean NOT NULL DEFAULT false,

```

```

user_id uuid REFERENCES auth.users(id)
);

-- Enforce uniqueness: one row per card per binder
CREATE UNIQUE INDEX card_ownership_binder_card_uidx
ON card_ownership (binder_id, card_id);

-- Fast lookup for the binder page card list
CREATE INDEX card_ownership_binder_id_idx
ON card_ownership (binder_id);

-- Fast lookup and sort for canonical order
CREATE INDEX card_ownership_collector_num_idx
ON card_ownership (binder_id, collector_number ASC);

```

Entity relationships

Relationship	Cardinality	Enforcement
binders -> card_ownership	1 : N	FK + ON DELETE CASCADE
binder + card_id	1 : 1 (unique pair)	UNIQUE INDEX (binder_id, card_id)
auth.users -> binders	1 : N (future)	FK on user_id (inactive)

Row Level Security (RLS) — forward compatibility

RLS is not enabled in the current phase. The schema is designed so that enabling it requires zero DDL changes — only policy creation. When auth is introduced, the following policies will be applied:

RLS POLICIES — applied when auth is introduced

```

-- Enable RLS (one-time, when auth lands)
ALTER TABLE binders ENABLE ROW LEVEL SECURITY;
ALTER TABLE card_ownership ENABLE ROW LEVEL SECURITY;

-- Users can only see and modify their own binders
CREATE POLICY binders_owner ON binders
USING (user_id = auth.uid());

-- card_ownership inherits user isolation via binder relationship
CREATE POLICY card_ownership_owner ON card_ownership
USING (user_id = auth.uid());

```

Progress counter query

The progress counter displayed in both the Home card and the binder header is derived from a single aggregation query per binder:

PROGRESS QUERY

```
SELECT
  COUNT(*) AS total,
  COUNT(*) FILTER (WHERE owned) AS owned_count,
  COUNT(*) FILTER (WHERE NOT owned) AS missing_count
FROM card_ownership
WHERE binder_id = $1;
```

ON DELETE CASCADE note: deleting a binder row automatically removes all associated card_ownership rows via the foreign key cascade. No application-level cleanup is required. No orphan rows are possible.

Collector number sort note: collector_number is stored as text because Scryfall uses values like '123a', '123b' for variants. Sorting must use ORDER BY collector_number::int or a natural sort function to avoid lexicographic ordering artifacts (e.g. '10' < '9' in text sort). This must be handled at query time.

Scope & Future Work

Out of scope — deliberate decisions

The items below were evaluated and deliberately excluded from this phase. They are not oversights — they are choices.

User login & registration	System operates without auth. Schema is already auth-ready.
Binder sharing	No public links, no external view mode.
Manual card reordering	Canonical API order prevails. User reordering is a future feature.
Card copy quantity	One card is one card. No multiplicity tracking.
Additional card metadata	No foil flag, no acquisition date, no market value.
Offline mode	App requires connection for both Scryfall API and Supabase.
Data backup	Out of scope. User is responsible for their own data in this phase.
Card artwork	Cards display name and collector number only. Art is a future feature.

Forward compatibility built in at zero cost

Several decisions today were made with tomorrow in mind:

- user_id on both tables — auth lands without schema migration
- Supabase as the database — RLS is natively supported, multi-user isolation is one ALTER away
- collector_number stored — user reordering can be added with a separate sort_order column without touching existing data
- set_code and set_name stored separately — human-readable name is available without re-calling the API
- Unique index on (binder_id, card_id) — prevents duplicate ownership rows even if the client calls the API twice

This document must be consulted before any implementation decision is made. If a new feature does not fit the business logic described here, the conversation starts by revisiting the manifesto — not by opening the editor.